House Price Prediction

In Pune, there can be seen rapid growth in Real Estate because of increasing urbanization, IT sector expansion, and demand for residential properties. Predicting prices of the house based on the location will help them get the idea about approximate prices of the residence in Pune.

In this model, I have used the dataset available on Kaggle. Using a cleaned dataset of residential property listings, this project involves data preprocessing, exploratory data analysis (EDA), feature engineering, and model training. Multiple regression models were evaluated. The final model provides reliable price estimates that can be used in real estate advisory tools or platforms.

(NOTE: This project is only for educational purpose this data does not include real world data though designed to mirroring the structure and attributes commonly found in housing price prediction dataset.)

Dataset Description:

1. Number of Entries: 100000
2. File Format: CSV
3. Data Attributes
    - ID
    - Area
    - Square feet
    - Number of Bedrooms
    - Number of Bathrooms
    - Year Built
    - Has garage
    - Price

Usage:

This dataset can be used for various machine learning tasks, including:

Regression Analysis: Predicting house prices based on features like area, square footage, number of bedrooms, and other attributes.
Exploratory Data Analysis (EDA): Understanding the distribution of house prices across different localities and features.
Feature Engineering: Creating and testing new features to improve predictive models.

Data Preprocessing

Using Python library that is Pandas to pre-process the data such as read data in csv files, get info of data that is column and rows, Data Cleaning: - handling missing values by either dropping the null values of aggregating it using mean/ media/ mode, handling duplicate values by dropping them, changing the names of columns or rows if needed.

EDA

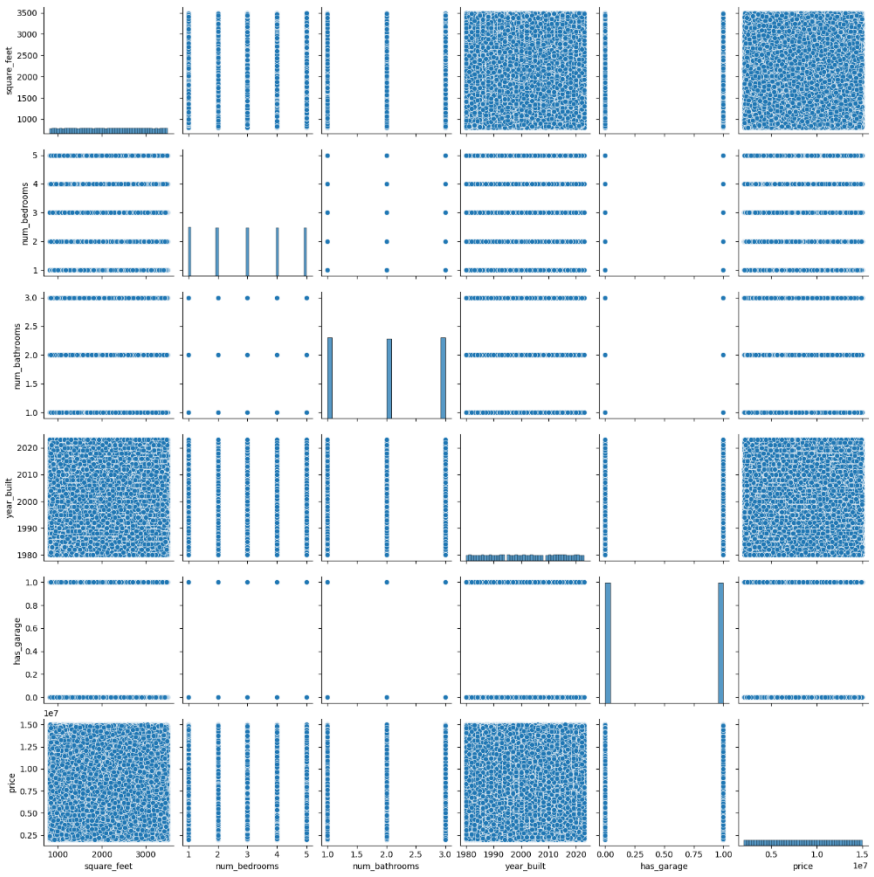
Performing Exploratory Data analysis for visualization of the data and knowing the data and their relation with each other. This helps to understand the data in better way. Visualizing data using Python Libraries such as Matplotlib for basic histogram, bar graph, line graph, Heatmap for correlation and Seaborn for pairplot (pairplot helps to visualize data in pair for like matrix and we can visualize and understand relation between two different variables.)

Here, first used describe function to know the values such as mean, inter quartile range, standard deviation, min and max value.

Second, Visualizations:

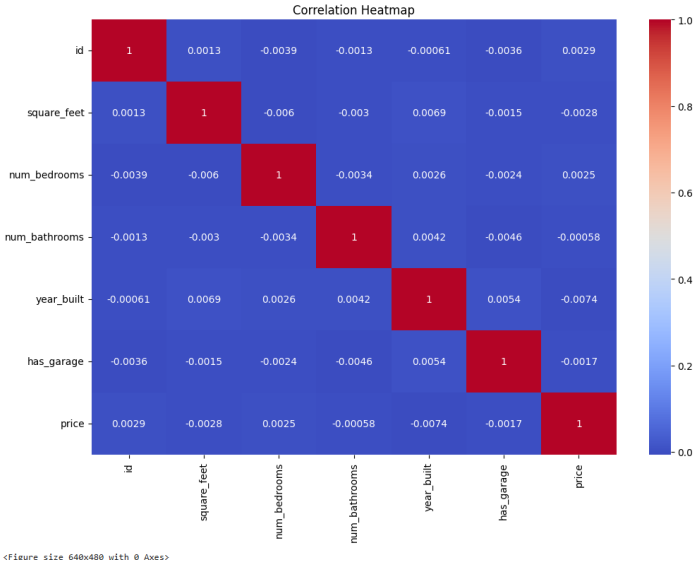
1. Pairplot analysis:
   The Pairplot analysis also known as scattered matrix that shows pairwise relationship between data's variables. Each subplots compares to features one on the X-axis and second on the Y-axis and diagonal plots show the distribution (histogram) of each individual feature.
   Observations:
   1. Most Features are discrete
   2. Very little visible trend with price
   3. Horizontal Banding in price

   a. The price values are tightly packed in a range, which might indicate:
      1. Outliers
      2. Data preprocessing needed (like log transformation)
      3. Clipping or artificial price capping



2. Heatmap Analysis:
   It visualizes the correlation matrix between numerical features in the data to understand the linear relationship between data.
   Correlation ranges between -1 to 1.
   - +1 – Perfect Positive Correlation (as one increases, so does the other)
   - -1 – Perfect Negative Correlation (as one increases, other decreases)
   - 0 – No Linear Relationship

   Diagonal value = 1: Every variable is perfectly correlated with itself



Interpretation of Heatmap:

Feature Pair	**Correlation with price**	**Insight**
square_feet vs. price	-0.0028	Almost no correlation (very weak negative) this is unusual and might suggest: data quality issue, outliers, or poorly scaled values. Normally, square footage correlates positively with price.
num_bedrooms vs. price	0.0025	Very weak positive again, surprisingly low; likely suggests weak or nonlinear relationships.
year_built vs. price	-0.0074	Slight negative correlation newer homes are not clearly priced higher. Could be due to mixed age across locations.
has_garage vs. price	-0.0017	Essentially no correlation.
id vs. anything	Ignore – id is a unique identifier and should be dropped from modeling.	

- Most features are very weakly correlated with price
- Non-linear relationships (e.g., price might increase exponentially with size).
- Data quality issues (missing values, encoding problems).
- Important features are missing (e.g., location, furnishing_status, etc., are likely more influential).
- Price has been normalized or obfuscated.

3. Skewness:
   It helps to understand whether the data is symmetrically distributed or leaning to one side. Types of skew:
   - Positive skew – long tail on right side (Right skew). Value > 0
   - Negative skew – long tail on left side (Left skew). Value < 0
   - Zero skew – Symmetrical (like a normal distribution). A bell Like curve . Value ~ 0

```
numeric_data.skew()
```

```
id               0.000000
square_feet     -0.002193
num_bedrooms     0.002492
num_bathrooms    0.000846
year_built      -0.002209
has_garage      -0.002920
price            0.001547
dtype: float64
```

1. Skewness Values are Near Zero
   - All values are very close to 0 (between -0.003 and 0.003), meaning the data distributions are nearly symmetrical.
2. Feature-wise Skewness:
   - square_feet:- 0.002193 -- Slightly left-skewed, but nearly symmetric.
   - num_bedrooms:- 0.002492 -- Slightly right-skewed, almost symmetric.
   - num_bathrooms:-0.000846 -- Very close to zero, practically symmetric.
   - year_built:- 0.002209 -- Slight left skew, not significant.
   - has_garage:- 0.002920 -- Slight left skew, but also negligible.
   - price:- 0.001547 -- Almost perfectly symmetric, great for regression.
3. No Transformation Needed
   - Since none of the features are strongly skewed (skewness > ±1), you don't need to apply transformations like log, square root, or Box-Cox.
4. Modelling Friendly
   - The near-symmetric distribution of price and other features is ideal for linear models (e.g., Linear Regression), which assume normally distributed residuals.
5. Good Data Quality
   - This skewness report suggests that data is well-distributed, which can lead to more stable and accurate model performance.

Feature Engineering

Adding two columns one, named by “age_of_house” for knowing the age of the property. This will help to understand how the age of the property influences the price of the property.

	id	area	square_feet	num_bedrooms	num_bathrooms	year_built	has_garage	price	age_of_house
0	1	Viman Nagar	1521	1	1	2016	0	4315847	9
1	2	Kalyani Nagar	2957	1	2	2003	1	12861115	22
2	3	Pimpri-Chinchwad	2816	3	2	1993	0	8615274	32
3	4	Kalyani Nagar	2869	3	1	2000	1	5360008	25
4	5	Kalyani Nagar	1285	3	3	2012	0	6412778	13

Second, “price_per_squareft” this will help to get the idea about price per square feet of the property.

```
def age_of_house(year_built):
    return 2025 - year_built

data["age_of_house"] = data['year_built'].apply(age_of_house)
```

```
data.head()
```

	id	area	square_feet	num_bedrooms	num_bathrooms	year_built	has_garage	price	age_of_house
0	1	Viman Nagar	1521	1	1	2016	0	4315847	9
1	2	Kalyani Nagar	2957	1	2	2003	1	12861115	22
2	3	Pimpri-Chinchwad	2816	3	2	1993	0	8615274	32
3	4	Kalyani Nagar	2869	3	1	2000	1	5360008	25
4	5	Kalyani Nagar	1285	3	3	2012	0	6412778	13

Model Training

The training of the house price prediction model carried out in a structured and sequential manner to ensure data readiness, model accuracy, and robust evaluation. The process began by importing all essential Python libraries. These included libraries for data manipulation (pandas, numpy), model training and evaluation (scikit-learn), and visualization (matplotlib, seaborn).

Data Cleaning and Preparation

Before model training, the dataset was carefully reviewed for irrelevant or unnecessary information.
Columns that did not contribute meaningfully to the prediction task, such as unique identifiers or duplicate data, were removed to simplify the model and improve performance. Categorical variables, particularly the 'area' column, were converted into numerical format using one-hot encoding.
This step ensured that machine learning models could interpret these values effectively. Additional features such as ‘age_of_house’ and ‘price_per_squareft’ were also created through feature engineering to enrich the dataset.

Splitting the Dataset

To assess the model’s performance accurately, the data was split into two parts: 80% for training and 20% for testing.
This split allowed the model to learn patterns from the majority of the data while being evaluated on unseen samples.
The shape of both the training and test sets was printed to confirm the correct distribution of samples.

Outlier Detection

Outliers were examined using the interquartile range (IQR) method.
This statistical technique helps identify the spread of the middle 50% of the data and highlights any extreme values that could negatively affect model training.
By calculating the IQR for numerical features, we gained insight into which values might need to be removed or treated to prevent skewed results.

Model Training and Prediction

Once the data was prepared, different regression models were trained on the training set.
These included linear regression, ridge regression, and lasso regression.
Each model was trained using the same input features, allowing for a fair comparison of performance.
After training, predictions were generated on the test set.

Model Evaluation

To evaluate model performance, we used two key metrics:

- Root Mean Squared Error (RMSE)
- R-squared (R^2).

RMSE measures the average difference between actual and predicted values, while R^2 indicates how well the model explains the variance in the target variable. These metrics were used to compare the accuracy of different models and assess whether they were suitable for real-world prediction tasks. A lower RMSE and a higher R^2 score were preferred, as they indicated better model accuracy and fit.